

SDD — Visualizador Web Didáctico de Estructuras de Datos

1. Información general del proyecto

Nombre del proyecto: Visualizador Web de Estructuras de Datos

Versión: 0.1.0

Estado: Especificación inicial para implementación del módulo de estructuras secuenciales

Tecnologías: Python, Flask, HTML5, CSS3, JavaScript, Jinja2

Tipo de proyecto: Aplicación web didáctica

Público objetivo: Estudiantes, docentes y monitores de estructuras de datos

Descripción general

El sistema será una aplicación web didáctica que permitirá a estudiantes aprender el comportamiento de diferentes estructuras de datos mediante visualizaciones interactivas, ejecución de operaciones y ayudas conceptuales.

La aplicación estará organizada en tres módulos principales:

1. Estructuras secuenciales.
2. Estructuras jerárquicas.
3. Grafos.

La primera fase implementará únicamente el módulo de **estructuras secuenciales**, usando los TAD suministrados inicialmente en Python.

Objetivo del sistema

Diseñar e implementar una plataforma web que permita seleccionar una estructura de datos, ejecutar operaciones sobre ella, visualizar su estado interno y consultar ayuda didáctica asociada.

Alcance inicial

La primera versión debe incluir los siguientes TAD:

- Pila.
 - Cola.
 - Cola de prioridad.
 - Lista enlazada.
 - Lista circular.
 - Sublista.
 - Excepciones comunes.
-

2. Visión del sistema

Problema que resuelve

Los estudiantes suelen aprender estructuras de datos de forma abstracta, leyendo código o diagramas estáticos. El sistema busca facilitar el aprendizaje mediante interacción visual, mensajes didácticos y ejecución controlada de operaciones.

Usuarios objetivo

- Estudiantes de programación y estructuras de datos.
- Docentes que necesitan apoyar sus explicaciones.
- Monitores académicos.
- Desarrolladores educativos.

Beneficios esperados

- Comprensión visual del comportamiento de las estructuras.
- Separación clara entre implementación lógica y representación gráfica.
- Reutilización de TAD existentes.
- Preparación para futuras implementaciones en lenguaje C.
- Base extensible para árboles, grafos y algoritmos.

Contexto de uso

El sistema será usado desde un navegador web. Inicialmente se ejecutará localmente o en un servidor académico mediante Flask.

3. Requisitos funcionales

ID	Requisito	Descripción	Prioridad	Actor
RF-001	Página principal	Mostrar descripción del proyecto y acceso a módulos	Alta	Estudiante
RF-002	Módulo secuencial	Permitir acceder al módulo de estructuras secuenciales	Alta	Estudiante
RF-003	Selección de estructura	Permitir seleccionar pila, cola, cola de prioridad, lista enlazada, lista circular o sublista	Alta	Estudiante
RF-004	Visualización de estado	Mostrar gráficamente el estado actual de la estructura	Alta	Estudiante
RF-005	Ejecución de operaciones	Permitir ejecutar operaciones válidas según el TAD seleccionado	Alta	Estudiante
RF-006	Mensajes didácticos	Mostrar explicación del resultado de cada operación	Alta	Estudiante

ID	Requisito	Descripción	Prioridad	Actor
RF-007	Manejo de errores	Mostrar errores comprensibles cuando una operación no sea válida	Alta	Estudiante
RF-008	Ayuda por módulo	Proveer ayuda general para estructuras secuenciales	Alta	Estudiante
RF-009	Ayuda por estructura	Proveer explicación individual para cada TAD	Alta	Estudiante
RF-010	Plantillas reutilizables	Usar header, footer, navbar y layout base reutilizables	Alta	Desarrollador
RF-011	Separación lógica/visual	Separar TAD, servicios, rutas, plantillas y JavaScript	Alta	Desarrollador
RF-012	Estado de sesión	Mantener temporalmente el estado de la estructura activa	Media	Estudiante
RF-013	Preparación para C	Diseñar la app para soportar TAD implementados en C en una fase futura	Alta	Desarrollador

4. Requisitos no funcionales

ID	Tipo	Descripción
RNF-001	Documentación	Todo el código debe estar documentado según estándares del lenguaje
RNF-002	Python	El código Python debe cumplir PEP8, usar docstrings y type hints cuando sea razonable
RNF-003	JavaScript	El código JS debe separar eventos, renderizado y comunicación con backend
RNF-004	HTML	Las plantillas deben usar HTML semántico y Jinja2
RNF-005	CSS	Los estilos deben dividirse en base, layout, componentes y visualización
RNF-006	Modularidad	Cada estructura debe estar aislada mediante adaptadores
RNF-007	Mantenibilidad	Debe ser sencillo agregar árboles, grafos y nuevos TAD
RNF-008	Seguridad	Validar entradas y evitar ejecución de código arbitrario
RNF-009	Usabilidad	La interfaz debe ser comprensible para estudiantes principiantes
RNF-010	Rendimiento	El sistema debe funcionar con estructuras pequeñas y medianas de uso didáctico
RNF-011	Extensibilidad	La integración futura con C no debe afectar la UI
RNF-012	Trazabilidad	Cada requisito debe mapearse con casos de uso, módulos y pruebas

5. Casos de uso

CU-001 — Acceder a la página principal

Actor: Estudiante

Descripción: El usuario abre la aplicación web.

Precondiciones: El servidor Flask está activo.

Flujo principal:

1. El usuario ingresa a la URL principal.
2. El sistema muestra el nombre del proyecto.
3. El sistema muestra los módulos disponibles.
4. El sistema indica que estructuras secuenciales está activo y los demás módulos están pendientes.

Postcondición: El usuario puede seleccionar el módulo secuencial.

CU-002 — Seleccionar módulo secuencial

Actor: Estudiante

Descripción: El usuario entra al módulo de estructuras secuenciales.

Precondiciones: La página principal está cargada.

Flujo principal:

1. El usuario selecciona "Estructuras secuenciales".
2. El sistema muestra una lista de estructuras disponibles.
3. Cada estructura se muestra como una tarjeta con descripción breve.

Postcondición: El usuario puede seleccionar una estructura específica.

CU-003 — Seleccionar una estructura

Actor: Estudiante

Descripción: El usuario selecciona una estructura para interactuar.

Precondiciones: El usuario está en el módulo secuencial.

Flujo principal:

1. El usuario selecciona una estructura.
2. El sistema crea o recupera el estado de esa estructura.
3. El sistema muestra operaciones disponibles.
4. El sistema muestra el estado visual inicial.

Postcondición: La estructura queda activa para ejecutar operaciones.

CU-004 — Insertar elemento

Actor: Estudiante

Descripción: El usuario agrega un elemento a una estructura.

Precondiciones: Existe una estructura activa.

Flujo principal:

1. El usuario ingresa un valor.
2. El usuario presiona el botón de inserción correspondiente.
3. El sistema valida el dato.
4. El sistema ejecuta la operación mediante el adaptador.
5. El sistema actualiza el estado visual.
6. El sistema muestra una explicación didáctica.

Postcondición: La estructura contiene el nuevo elemento.

CU-005 — Eliminar elemento

Actor: Estudiante

Descripción: El usuario elimina un elemento según las reglas de la estructura.

Precondiciones: La estructura no debe estar vacía.

Flujo principal:

1. El usuario solicita eliminar o extraer.
2. El sistema valida la operación.
3. El adaptador ejecuta la operación real del TAD.
4. El sistema actualiza la visualización.
5. El sistema muestra el elemento eliminado.

Flujo alternativo: Si la estructura está vacía, el sistema muestra un mensaje didáctico.

Postcondición: La estructura queda actualizada.

CU-006 — Consultar ayuda

Actor: Estudiante

Descripción: El usuario consulta explicación de un módulo o estructura.

Precondiciones: El usuario está en la aplicación.

Flujo principal:

1. El usuario selecciona ayuda.
2. El sistema muestra explicación conceptual.
3. El sistema muestra operaciones principales.
4. El sistema muestra ejemplos y errores comunes.

Postcondición: El usuario comprende mejor la estructura consultada.

6. Modelo de datos

6.1 Diccionario de datos

Entidad: StructureSession

Atributo	Tipo	Descripción
session_id	string	Identificador de sesión
module	string	Módulo activo
structure_type	string	Tipo de estructura seleccionada
state	dict/list	Estado serializable de la estructura
last_operation	string	Última operación ejecutada
last_result	dict/string	Resultado de la última operación
errors	list	Errores recientes

Entidad: StructureDefinition

Atributo	Tipo	Descripción
id	string	Identificador interno
name	string	Nombre visible
description	string	Descripción didáctica
operations	list	Operaciones disponibles
help_content	dict	Contenido de ayuda
adapter_class	string	Adaptador asociado

Entidad: OperationDefinition

Atributo	Tipo	Descripción
id	string	Identificador interno
name	string	Nombre visible
input_schema	dict	Parámetros requeridos
description	string	Explicación didáctica
preconditions	list	Condiciones previas
postconditions	list	Condiciones posteriores

7. Arquitectura del sistema

7.1 Estilo arquitectónico

El sistema usará una arquitectura web modular por capas:

```
Usuario
  ↓
Frontend HTML/CSS/JavaScript
  ↓
Rutas Flask / Controladores
  ↓
Servicios de aplicación
  ↓
Adaptadores de estructuras
  ↓
TAD Python actuales
  ↓
Futuras implementaciones en C
```

7.2 Capas principales

Capa de presentación

Responsable de:

- Renderizar plantillas.
- Mostrar formularios.
- Dibujar estructuras.
- Mostrar ayuda.
- Ejecutar animaciones.

Debe contener:

```
templates/
static/css/
static/js/
```

Capa de rutas Flask

Responsable de:

- Recibir solicitudes HTTP.
- Validar datos básicos.
- Invocar servicios.
- Retornar HTML o JSON.

No debe contener lógica interna de los TAD.

Capa de servicios

Responsable de:

- Coordinar operaciones.
- Manejar estado de sesión.
- Traducir respuestas para la UI.
- Capturar excepciones.
- Preparar mensajes didácticos.

Capa de adaptadores

Responsable de normalizar el acceso a las estructuras.

Adaptadores previstos:

- StackAdapter.
- QueueAdapter.
- PriorityQueueAdapter.
- LinkedListAdapter.
- CircularListAdapter.
- SubListAdapter.

Cada adaptador debe ofrecer una interfaz homogénea para la aplicación, aunque internamente cada TAD tenga nombres de métodos distintos.

Capa de dominio

Contiene los TAD originales. Esta capa debe mantenerse independiente de Flask, HTML, CSS y JavaScript.

8. Estructura de carpetas propuesta

```
visualizador_estructuras/  
|  
├─ app/  
|   ├── __init__.py  
|   └─ routes/  
|       ├── __init__.py  
|       ├── main_routes.py  
|       ├── sequential_routes.py  
|       └─ help_routes.py  
|  
└─ services/  
    ├── __init__.py  
    ├── structure_service.py  
    ├── session_service.py  
    └─ help_service.py
```



```
├── adapters/
│   ├── __init__.py
│   ├── base_adapter.py
│   ├── stack_adapter.py
│   ├── queue_adapter.py
│   ├── priority_queue_adapter.py
│   ├── linked_list_adapter.py
│   ├── circular_list_adapter.py
│   └── sublist_adapter.py
├── domain/
│   ├── __init__.py
│   ├── sequential/
│   │   ├── __init__.py
│   │   ├── pila.py
│   │   ├── cola.py
│   │   ├── cola_prioridad.py
│   │   ├── lista_enlazada.py
│   │   ├── lista_circular.py
│   │   ├── sublista.py
│   │   └── exceptions.py
│   ├── hierarchical/
│   │   └── README.md
│   ├── graphs/
│   │   └── README.md
├── templates/
│   ├── base.html
│   ├── partials/
│   │   ├── header.html
│   │   ├── footer.html
│   │   ├── navbar.html
│   │   ├── messages.html
│   │   └── help_panel.html
│   ├── pages/
│   │   ├── index.html
│   │   ├── sequential.html
│   │   └── structure_view.html
│   ├── help/
│   │   └── sequential_help.html
└── static/
    ├── css/
    │   ├── base.css
    │   ├── layout.css
    │   └── components.css
```

```
|
|   |   └─ visualizer.css
|   |
|   └─ js/
|       ├── api_client.js
|       ├── visualizer.js
|       ├── sequential_visualizer.js
|       ├── animations.js
|       └─ helpers.js
|
|   └─ img/
|
└─ tests/
    ├── test_stack_adapter.py
    ├── test_queue_adapter.py
    ├── test_priority_queue_adapter.py
    ├── test_linked_list_adapter.py
    ├── test_circular_list_adapter.py
    ├── test_sublist_adapter.py
    └─ test_routes.py
|
└─ docs/
    ├── sdd.md
    ├── architecture.md
    ├── user_guide.md
    └─ future_c_integration.md
|
└─ config.py
└─ run.py
└─ requirements.txt
└─ README.md
└─ CHANGELOG.md
```

9. Contratos de interfaz UI/UX

Pantalla principal

Debe incluir:

- Título del sistema.
- Breve explicación.
- Tarjetas de módulos.
- Estado de disponibilidad:
- Secuenciales: activo.
- Jerárquicas: próximamente.
- Grafos: próximamente.

Pantalla del módulo secuencial

Debe incluir:

- Lista de estructuras.
- Descripción corta de cada una.
- Botón "Explorar".
- Botón "Ayuda del módulo".

Pantalla de visualización

Header / Navbar	
Nombre de estructura	
Descripción breve	
Panel de operaciones	Visualización gráfica
Explicación paso a paso	
Resultado / errores	
Footer	

Contrato visual de estructura

Cada visualización debe recibir un estado serializable similar a:

```
{
  "structure": "stack",
  "items": ["A", "B", "C"],
  "metadata": {
    "top": "C",
    "size": 3,
    "is_empty": false
  },
  "last_operation": {
    "name": "push",
    "value": "C",
    "status": "success",
    "message": "El elemento C fue insertado en la cima."
  }
}
```

10. Reglas de negocio

ID	Regla	Descripción
RB-001	Operaciones válidas	Solo se deben mostrar operaciones soportadas por el TAD seleccionado
RB-002	Estructura vacía	No se deben ejecutar operaciones de extracción o consulta sobre estructuras vacías
RB-003	Mensajes didácticos	Todo error técnico debe traducirse a lenguaje comprensible para estudiantes
RB-004	Dominio independiente	Los TAD no deben importar Flask ni depender de la interfaz
RB-005	Estado serializable	El backend debe entregar al frontend datos JSON serializables
RB-006	Contrato estable	Los nombres internos de operaciones deben permanecer estables para pruebas e integración futura
RB-007	Integración futura con C	El cambio de implementación Python a C debe ocurrir detrás de la capa de adaptadores
RB-008	No inventar operaciones	Si una operación no existe en el TAD, debe marcarse como pendiente
RB-009	Validación de entradas	Todo dato ingresado por el usuario debe validarse antes de ejecutar operaciones

11. Operaciones iniciales por TAD

Pila

Operaciones esperadas:

- `apilar(dato)` / `push(dato)`.
- `desapilar()` / `pop()`.
- `cima()` / `peek()`.
- `limpiar()`.
- `vacía()`.
- `tamaño()`.
- `a_lista()`.

Cola

Operaciones esperadas:

- `encolar(dato)` / `enqueue(dato)`.
- `desencolar()` / `dequeue()`.
- `frente()`.
- `final()`.

- limpiar().
- vacia().
- tamano().
- a_lista().

Cola de prioridad

Operaciones esperadas:

- encolar(dato, prioridad).
- desencolar().
- frente().
- vacia().
- tamano().
- limpiar().
- a_lista().

Lista enlazada

Operaciones esperadas:

- insertar_inicio(dato).
- insertar_final(dato).
- insertar_posicion(posicion, dato).
- eliminar_inicio().
- eliminar_final().
- eliminar_posicion(posicion).
- eliminar_primero(dato).
- buscar_posiciones(dato).
- invertir().
- primero().
- ultimo().
- limpiar().
- vacia().
- tamano().
- a_lista().

Lista circular

Operaciones esperadas:

- insertar_inicio(dato).
- insertar_final(dato).
- eliminar_inicio().
- buscar_posiciones(dato).
- limpiar().
- vacia().
- tamano().
- a_lista().

Sublista

Operaciones esperadas:

- `insertar_padre(dato)`.
 - `buscar_padre(dato)`.
 - `insertar_hijo(padre, hijo)`.
 - `eliminar_padre(dato)`.
 - `hijos_de(padre)`.
 - `a_diccionario()`.
 - `limpiar()`.
-

12. Especificación de algoritmos

Algoritmo general para ejecutar una operación

Entrada:

- Tipo de estructura.
- Operación solicitada.
- Parámetros del usuario.
- Estado actual de sesión.

Proceso:

1. Validar que la estructura exista.
2. Validar que la operación sea permitida.
3. Validar parámetros.
4. Recuperar instancia de estructura desde sesión.
5. Invocar el adaptador correspondiente.
6. Capturar errores del TAD.
7. Convertir el resultado a formato didáctico.
8. Serializar nuevo estado.
9. Retornar respuesta a la UI.

Salida:

- Estado actualizado.
- Resultado de operación.
- Mensaje didáctico.
- Datos para visualización.

Algoritmo de renderizado visual

Entrada: Estado serializado de la estructura.

Proceso:

1. Limpiar contenedor visual.
2. Identificar tipo de estructura.

3. Convertir elementos en nodos visuales.
4. Dibujar relaciones si aplica.
5. Resaltar último elemento afectado.
6. Mostrar etiquetas como top, front, rear o prioridad.
7. Mostrar explicación de la operación.

Salida: Visualización actualizada en pantalla.

13. API / servicios

Rutas web sugeridas

Endpoint	Método	Descripción
/	GET	Página principal
/secuenciales	GET	Página del módulo secuencial
/secuenciales/<structure_id>	GET	Página de visualización de una estructura
/api/secuenciales/<structure_id>/operation	POST	Ejecutar operación sobre una estructura
/api/secuenciales/<structure_id>/state	GET	Obtener estado actual serializado
/api/secuenciales/<structure_id>/reset	POST	Reiniciar estructura
/ayuda/secuenciales	GET	Ayuda general del módulo
/ayuda/secuenciales/<structure_id>	GET	Ayuda específica por estructura

Servicio: StructureService

Responsabilidades:

- Listar estructuras disponibles.
- Crear estructuras.
- Ejecutar operaciones.
- Consultar estado.
- Reiniciar estructura.

Métodos conceptuales:

```
get_available_structures()
create_structure(structure_type)
execute_operation(structure_type, operation, payload)
get_structure_state(structure_type)
reset_structure(structure_type)
```

Servicio: SessionService

Responsabilidades:

- Guardar estado temporal.
- Recuperar estructura activa.
- Limpiar sesión.
- Serializar y deserializar datos.

Servicio: HelpService

Responsabilidades:

- Obtener ayuda por módulo.
- Obtener ayuda por estructura.
- Obtener descripción de operaciones.

Contrato de adaptador base

Cada adaptador debe cumplir una interfaz común:

```
create()
execute(operation_name, payload)
to_visual_state()
reset()
get_supported_operations()
```

14. Trazabilidad

RF	Caso de uso	Módulo	Prueba
RF-001	CU-001	Main Routes	test_home_page_loads
RF-002	CU-002	Sequential Routes	test_sequential_module_loads
RF-003	CU-003	Sequential Routes	test_structure_selection
RF-004	CU-003, CU-004, CU-005	Visualizer JS / Templates	test_visual_state_response
RF-005	CU-004, CU-005	Adapters	test_adapter_operations
RF-006	CU-004, CU-005	StructureService	test_operation_message
RF-007	CU-005	Exceptions / Services	test_empty_structure_error
RF-008	CU-006	HelpService	test_module_help_available
RF-009	CU-006	HelpService	test_structure_help_available
RF-010	CU-001 a CU-006	Templates	test_base_template_blocks

RF	Caso de uso	Módulo	Prueba
RF-011	Todos	Domain / Services / Adapters	test_no_flask_dependency_in_domain
RF-012	CU-003 a CU-005	SessionService	test_session_state_persistence
RF-013	Futuro	Adapter Layer	test_adapter_contract_stability

15. Plan de pruebas

Pruebas unitarias

Deben cubrir:

- Adaptadores.
- Servicios.
- Conversión a estado visual.
- Manejo de errores.
- Validaciones de entrada.

Pruebas de integración

Deben validar:

- Carga de rutas Flask.
- Flujo completo de selección de estructura.
- Ejecución de operación.
- Respuesta JSON/HTML.
- Persistencia en sesión.

Pruebas de interfaz

Deben validar:

- Carga de plantillas.
- Botones visibles.
- Mensajes de ayuda.
- Visualización actualizada tras operaciones.

Pruebas didácticas

Deben validar que:

- Cada operación tenga explicación.
- Los errores sean comprensibles.
- Cada estructura tenga ayuda.
- El usuario pueda entender el estado actual.

16. Criterios de aceptación

La primera versión será aceptada cuando:

1. La aplicación Flask cargue correctamente.
 2. Exista una página principal con navegación.
 3. Exista el módulo de estructuras secuenciales.
 4. Se puedan seleccionar las estructuras del primer módulo.
 5. Cada estructura tenga una interfaz de operaciones.
 6. La lógica de los TAD esté separada de la interfaz.
 7. Las plantillas compartan header, footer y navegación.
 8. Cada estructura tenga ayuda didáctica.
 9. Los errores sean controlados y explicados.
 10. El código Python cumpla PEP8 y tenga docstrings.
 11. El frontend esté separado en archivos CSS y JavaScript.
 12. Exista una capa de adaptadores preparada para futuras implementaciones en C.
 13. Exista un conjunto inicial de pruebas.
 14. El agente no haya modificado innecesariamente los TAD originales sin justificación.
-

17. Gestión del proyecto

Metodología

Desarrollo basado en especificaciones, con implementación incremental por módulos.

Herramientas sugeridas

- Python 3.
- Flask.
- Pytest.
- HTML5.
- CSS3.
- JavaScript.
- Git.
- Agente IA de desarrollo.

Fases

Fase 1 — Base del proyecto

- Crear estructura Flask.
- Crear plantillas base.
- Configurar rutas principales.
- Integrar archivos estáticos.

Fase 2 — Dominio y adaptadores

- Ubicar TAD en `domain/sequential`.
- Crear adaptador base.
- Crear adaptadores por estructura.

- Normalizar estados visuales.

Fase 3 — Módulo secuencial

- Crear página del módulo.
- Crear página de visualización.
- Crear operaciones por estructura.
- Crear ayuda por estructura.

Fase 4 — Visualización

- Implementar renderizado inicial.
- Agregar animaciones simples.
- Mostrar etiquetas didácticas.

Fase 5 — Pruebas y documentación

- Agregar pruebas unitarias.
- Agregar pruebas de rutas.
- Documentar uso.
- Documentar arquitectura.
- Documentar preparación para C.

18. Integración con agentes de IA

18.1 Roles

Agente	Responsabilidad
Agente arquitecto	Mantener arquitectura, revisar trazabilidad y evitar acoplamiento indebido
Agente backend	Implementar Flask, servicios, adaptadores, sesión y pruebas backend
Agente frontend	Implementar plantillas, CSS, JS, visualizaciones y componentes reutilizables
Agente documental	Crear README, guía de usuario, ayudas didácticas y docstrings
Agente QA	Crear pruebas unitarias, integración y validación de criterios de aceptación

18.2 Prompt base para agente desarrollador

Actúa como un agente desarrollador senior especializado en Python, Flask, HTML5, CSS3, JavaScript y arquitectura limpia para aplicaciones educativas.

Contexto del proyecto:

Debes construir la primera versión de una aplicación web didáctica llamada "Visualizador Web de Estructuras de Datos". Su objetivo es permitir que estudiantes aprendan estructuras de datos mediante visualizaciones interactivas.

La aplicación tendrá tres módulos generales:

1. Estructuras secuenciales.
2. Estructuras jerárquicas.
3. Grafos.

En esta primera fase solo debes implementar el módulo de estructuras secuenciales.

TAD disponibles inicialmente:

- pila.py
- cola.py
- cola_prioridad.py
- lista_enlazada.py
- lista_circular.py
- sublista.py
- exceptions.py

Objetivo técnico:

Crear una aplicación Flask modular que permita seleccionar una estructura de datos secuencial, ejecutar operaciones sobre ella, visualizar su estado y mostrar ayuda didáctica. La lógica de los TAD debe estar completamente separada de la capa visual.

Restricciones obligatorias:

1. No mezclar lógica de estructuras con rutas Flask ni plantillas HTML.
2. No modificar los TAD originales salvo que sea estrictamente necesario y documentado.
3. No inventar operaciones que no existan en los TAD. Si una operación requerida no existe, marcarla como pendiente.
4. Todo código Python debe cumplir PEP8.
5. Todo módulo, clase y función Python debe tener docstrings.
6. Usar nombres claros y type hints cuando sea razonable.
7. No usar eval.
8. Validar entradas de usuario.
9. Todos los errores deben traducirse a mensajes didácticos.
10. La interfaz debe usar plantillas reutilizables con header, footer, navbar y layout base.
11. El frontend debe estar separado en HTML, CSS y JavaScript.
12. La arquitectura debe quedar preparada para que en una fase futura los TAD puedan implementarse en C.

Arquitectura esperada:

Implementa una arquitectura por capas:

- Capa de presentación:
 - templates/
 - static/css/
 - static/js/
- Capa de rutas:
 - app/routes/main_routes.py
 - app/routes/sequential_routes.py

- app/routes/help_routes.py
- Capa de servicios:
 - app/services/structure_service.py
 - app/services/session_service.py
 - app/services/help_service.py
- Capa de adaptadores:
 - app/adapters/base_adapter.py
 - app/adapters/stack_adapter.py
 - app/adapters/queue_adapter.py
 - app/adapters/priority_queue_adapter.py
 - app/adapters/linked_list_adapter.py
 - app/adapters/circular_list_adapter.py
 - app/adapters/sublist_adapter.py
- Capa de dominio:
 - app/domain/sequential/
 - Allí deben ubicarse los TAD suministrados.

Funcionalidades mínimas:

1. Página principal con descripción del proyecto.
2. Navegación hacia el módulo de estructuras secuenciales.
3. Página del módulo secuencial con tarjetas para pila, cola, cola de prioridad, lista enlazada, lista circular y sublista.
4. Página de visualización para cada estructura.
5. Panel de operaciones dinámico según la estructura seleccionada.
6. Visualización gráfica del estado actual.
7. Mensajes didácticos después de cada operación.
8. Manejo de errores cuando la estructura esté vacía o la operación sea inválida.
9. Sección de ayuda para el módulo secuencial.
10. Ayuda específica para cada estructura.

Contrato de adaptador:

Cada adaptador debe implementar una interfaz común equivalente a:

- create()
- execute(operation_name, payload)
- to_visual_state()
- reset()
- get_supported_operations()

Criterios de aceptación:

La tarea se considera completa si:

1. La aplicación Flask ejecuta sin errores.
2. La página principal carga correctamente.
3. El módulo secuencial está disponible.
4. Cada estructura tiene una página de interacción.
5. Las operaciones disponibles se obtienen desde adaptadores.
6. El estado visual se actualiza después de cada operación.

7. El código de dominio no importa Flask.
8. Las plantillas reutilizan header, footer y navbar.
9. Existe ayuda didáctica del módulo y de cada estructura.
10. Existen pruebas iniciales.
11. El proyecto queda preparado para futura integración de TAD implementados en C.

Prohibiciones explícitas:

- No generar código monolítico.
- No poner lógica de estructuras en las rutas Flask.
- No poner lógica de negocio en JavaScript si debe estar en backend.
- No usar eval.
- No ignorar excepciones.
- No inventar métodos inexistentes en los TAD.
- No eliminar archivos TAD originales.
- No acoplar la UI directamente a una clase concreta del dominio.
- No implementar todavía estructuras jerárquicas, grafos ni Union-Find.

Formato de salida esperado:

1. Mostrar primero el árbol de carpetas creado.
2. Explicar brevemente la responsabilidad de cada carpeta.
3. Implementar los archivos necesarios.
4. Incluir comentarios y docstrings.
5. Incluir pruebas.
6. Incluir instrucciones para ejecutar:
 - creación de entorno virtual
 - instalación de dependencias
 - ejecución de Flask
 - ejecución de pruebas
7. Indicar claramente cualquier operación pendiente porque no exista en los TAD suministrados.

Antes de codificar:

Primero crea un plan de implementación detallado antes de desarrollar la aplicación. El plan debe incluir:

1. Análisis de los TAD suministrados, identificando clases, métodos, alias y excepciones reales.
2. Mapa de correspondencia entre operaciones de la interfaz y métodos reales de cada TAD.
3. Estructura final de carpetas y archivos a crear.
4. Orden de implementación por fases.
5. Componentes backend, frontend, servicios, adaptadores y pruebas que se van a construir.
6. Riesgos técnicos o dependencias faltantes detectadas.
7. Operaciones pendientes o no soportadas por los TAD reales.
8. Criterios de validación antes de iniciar la codificación.

Después de presentar el plan de implementación, desarrolla la aplicación siguiendo ese plan.

Analiza los TAD suministrados y extrae sus clases, métodos y excepciones reales. Usa esa información para construir los adaptadores. Si existe discrepancia entre esta especificación y los TAD reales, respeta los TAD reales y documenta la diferencia.

19. Control de versiones

Rama principal

main

Debe contener solo versiones estables.

Rama de desarrollo

develop

Para integración de funcionalidades.

Ramas sugeridas

feature/base-flask
feature/sequential-module
feature/structure-adapters
feature/ui-visualizer
feature/help-system
feature/tests

Convención de commits

feat: agrega módulo de estructuras secuenciales
fix: corrige validación de pila vacía
docs: agrega ayuda didáctica para cola
test: agrega pruebas para adaptador de lista enlazada
refactor: separa lógica de visualización

20. Decisiones arquitectónicas importantes

ID	Decisión	Justificación
DA-001	Separar dominio y presentación	Evita acoplamiento entre TAD y Flask

ID	Decisión	Justificación
DA-002	Usar adaptadores	Permite normalizar operaciones y preparar integración con C
DA-003	Serializar estado visual	Facilita comunicación backend/frontend
DA-004	Usar plantillas base	Evita duplicación en header, footer y navegación
DA-005	Priorizar didáctica	El objetivo principal es aprendizaje, no optimización extrema
DA-006	Mantener TAD originales	Reduce riesgo de alterar estructuras ya implementadas
DA-007	Marcar pendientes	Evita que agentes IA inventen operaciones no existentes